

Decorator Pattern

1. Problem Analysis

1.1 หากใช้ Inheritance เพียงอย่างเดียว จะเกิดปัญหาอะไรเมื่อมี Add-on จำนวนมาก

ตอบ ถ้าใช้ Inheritance อย่างเดียว จะต้องสร้างคลาสย่อยสำหรับกาแฟแต่ละแบบและ Add-on แต่ละรูปแบบ ทำให้จำนวนคลาสเพิ่มขึ้นมาก และถ้าลูกค้าสามารถเลือก Add-on ได้หลายอย่าง จะเกิดคลาสที่ต้องรองรับทุก Combination เช่น EspressoWithMilk, EspressoWithMilkSugar, AmericanoWithMilkSugar เป็นต้น ทำให้โค้ดซับซ้อน ดูแลรักษายาก และเพิ่ม Add-on ใหม่ได้ยาก

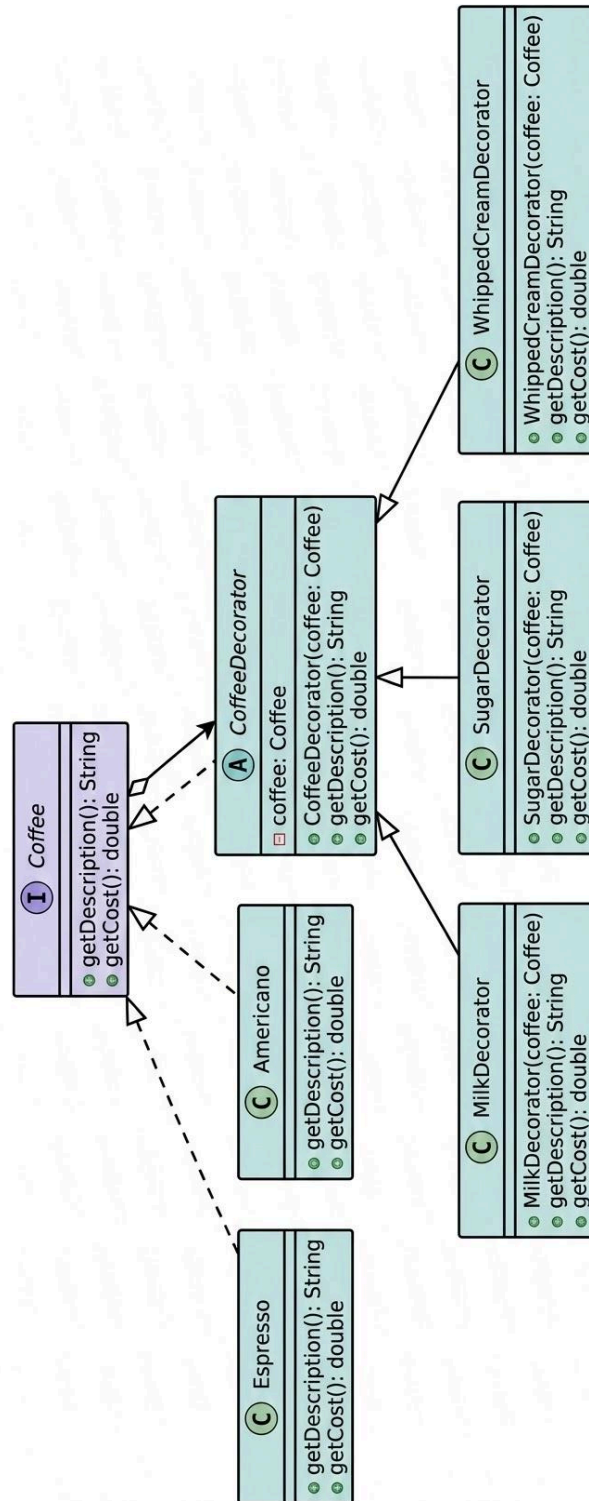
1.2 เพราะเหตุใด Decorator Pattern จึงเหมาะสมกับระบบนี้

ตอบ Decorator Pattern เหมาะกับระบบนี้เพราะลูกค้าสามารถเพิ่มส่วนประกอบเสริมให้กับกาแฟได้อย่างอิสระ เช่น Milk, Sugar และ Whipped Cream โดยสามารถนำ Decorator มาซ้อนกันหลายชั้นได้ โดยไม่ต้องสร้างคลาสใหม่สำหรับทุก Combination ของกาแฟและ Add-on

1.3 Decorator Pattern สนับสนุน Open-Closed Principle อย่างไร

ตอบ Decorator Pattern ทำให้สามารถเพิ่มความสามารถใหม่ให้กับระบบได้โดยการสร้าง Decorator ใหม่ โดยไม่จำเป็นต้องแก้ไขคลาสเดิม เช่น หากต้องการเพิ่ม Caramel สามารถสร้าง CaramelDecorator เพิ่มขึ้นมาได้ โดยไม่ต้องแก้ไข Espresso, Americano หรือ Decorator ตัวอื่นๆ

2. UML Design



3. Java Implementation

1. Coffee.java

กำหนดโครงสร้างพื้นฐานของกาแฟ โดยมี getDescription() สำหรับบอกรายละเอียดกาแฟ และ getCost() สำหรับบอกราคา เพื่อให้คลาสอื่น ๆ สามารถนำไปใช้งานร่วมกันได้

2. Espresso.java

เป็นคลาสกาแฟพื้นฐานชนิด Espresso ที่ทำตาม Coffee โดยกำหนดชื่อกาแฟเป็น Espresso และมีราคาเริ่มต้น \$2.00

3. Americano.java

เป็นคลาสกาแฟพื้นฐานชนิด Americano โดยกำหนดชื่อกาแฟเป็น Americano และมีราคาเริ่มต้น \$2.50

4. CoffeeDecorator.java

เป็นคลาส Decorator หลักที่ใช้เก็บวัตถุ Coffee เอาไว้ภายใน ทำให้สามารถนำกาแฟหรือ Decorator ตัวอื่นมาซ้อนกัน เพื่อเพิ่มส่วนประกอบต่าง ๆ ได้

5. MilkDecorator.java

ใช้สำหรับเพิ่ม Milk ให้กับกาแฟ โดยเพิ่มคำว่า Milk ในรายละเอียด และเพิ่มราคาอีก \$0.50

6. SugarDecorator.java

ใช้สำหรับเพิ่ม Sugar ให้กับกาแฟ โดยเพิ่มคำว่า Sugar ในรายละเอียด และเพิ่มราคาอีก \$0.20

7. WhippedCreamDecorator.java

ใช้สำหรับเพิ่ม Whipped Cream ให้กับกาแฟ โดยเพิ่มคำว่า Whipped Cream ในรายละเอียด และเพิ่มราคาอีก \$0.70

8. Main.java

เป็นคลาสสำหรับทดสอบการทำงานของระบบ โดยทดสอบกาแฟ 3 กรณี ได้แก่ Espresso, Espresso + Milk และ Americano + Milk + Sugar + Whipped Cream ตาม Test Case ที่กำหนดในโจทย์

4. Testing and Discussion

Test Case	Configuration	Description	Total Cost
1	new Espresso()	Espresso	\$2.00
2	new MilkDecorator(new Espresso())	Espresso + Milk	\$2.50
3	new WhippedCreamDecorator(new SugarDecorator(new MilkDecorator(new Espresso())))	Americano + Milk + Sugar + Whipped Cream	\$3.90

	Americano()))))		
--	-----------------	--	--

ผลการทดสอบโปรแกรม :

```

Test Case 1
Configuration: Espresso
Description: Espresso
Total Cost: $2.00

Test Case 2
Configuration: Espresso + Milk
Description: Espresso + Milk
Total Cost: $2.50

Test Case 3
Configuration: Americano + Milk + Sugar + Whipped Cream
Description: Americano + Milk + Sugar + Whipped Cream
Total Cost: $3.90

```

จากการทดสอบทั้ง 3 กรณี ระบบสามารถสร้างกาแฟพื้นฐานและเพิ่ม Add-on ได้ตามที่กำหนด รวมถึงสามารถซ้อน Decorator หลายชั้นและคำนวณราคารวมได้ถูกต้อง แสดงให้เห็นว่า Decorator Pattern สามารถรองรับการเพิ่มส่วนประกอบให้กับกาแฟได้ตามต้อง

5. Design Reflection

1. Decorator Pattern แตกต่างจาก Strategy Pattern อย่างไร

ตอบ Decorator Pattern ใช้สำหรับ เพิ่มความสามารถหรือพฤติกรรมให้ Object แบบสามารถซ้อนกันได้ เช่น การเพิ่ม Milk, Sugar และ Whipped Cream ให้กับกาแฟ ส่วน Strategy Pattern ใช้สำหรับ เปลี่ยนวิธีการทำงานหรือ Algorithm เช่น การเลือกวิธีคำนวณส่วนลดหลายรูปแบบ ดังนั้น Decorator เน้นการเพิ่มความสามารถให้ Object ส่วน Strategy เน้นการเลือกวิธีการทำงาน

2. หากมี Add-on ใหม่เพิ่มขึ้น 20 รายการ ระบบที่ใช้ Decorator Pattern จะมีข้อดีอย่างไร

ตอบ สามารถสร้าง Decorator ใหม่สำหรับ Add-on แต่ละรายการได้ โดยไม่ต้องแก้ไขคลาสกาแฟเดิม เช่น หากเพิ่ม Caramel ก็สร้าง **CaramelDecorator** และนำไปใช้งานร่วมกับ Decorator อื่นได้ทันที ทำให้ระบบขยายได้ง่ายและลดผลกระทบต่อโค้ดเดิม

3. หลักการ "Composition over Inheritance" ปรากฏอยู่ในส่วนใดของงานนี้

ตอบ หลักการนี้ปรากฏใน CoffeeDecorator ที่มีตัวแปร Coffee อยู่ภายใน และสามารถนำ Coffee หรือ Decorator ตัวอื่นมาประกอบกันได้ เช่น

```
Java
new SugarDecorator(
    new MilkDecorator(
        new Espresso()
    )
);
```

แทนที่จะสร้างคลาสลูกใหม่สำหรับทุก Combination ของกาแฟและ Add-on

4. ตัวอย่างระบบจริงที่สามารถใช้ Decorator Pattern ได้อีกอย่างน้อย 2 ระบบ

1. ระบบสั่งอาหาร

สามารถเพิ่ม Topping เช่น Cheese, Bacon หรือ Sauce ให้กับอาหารได้

2. ระบบสั่งพิซซ่า

สามารถเลือก Pizza พื้นฐานแล้วเพิ่ม Cheese, Ham, Mushroom หรืออื่น ๆ ได้ตามต้องการ